

كتالوج عملي لتجهيز + WSL + Ubuntu + Docker Containerlab

من الصفر حتى بناء شبكة Containers وتجربة Server/Client

إعداد: izzaldeen mansour

هذا الدليل كتالوج تدريبي عملي للطلاب. يبدأ من تثبيت WSL و Ubuntu داخل Windows، ثم تثبيت Docker و Containerlab، وبعدها إنشاء أول Topology وتشغيل نودات افتراضية وتجربة اتصال ورسائل بين Clients عبر Server.

ملاحظة مهمة: هذا الملف لا يحتوي على أي IP خاص بجهاز معين، ولا يتضمن إعدادات Port Forwarding من Windows إلى WSL. كل العناوين المستخدمة تعليمية داخل الالاب فقط.

خريطة الطريق المختصرة

المرحلة	الهدف	الناتج المتوقع
1	تثبيت WSL و Ubuntu	فتح Ubuntu Terminal داخل Windows
2	إعداد Ubuntu	تشغيل أوامر Linux الأساسية
3	تثبيت Docker	تشغيل Containers
4	تثبيت Containerlab	بناء شبكة من ملف YAML
5	إنشاء Topology	ثلاث نودات: server + client1 + client2
6	Deploy وفحص	نجاح Ping بين النودات
7	مثال رسائل	client1 يرسل إلى client2 عبر server

1. المتطلبات قبل البدء

- Windows 10 أو Windows 11 حديث.
- صلاحية Administrator على الجهاز.
- اتصال إنترنت أثناء التثبيت.
- تفعيل Virtualization من UEFI/BIOS إذا كانت غير مفعلة.
- مساحة تخزين مناسبة للتجارب والصور، ويفضل 10GB أو أكثر.

لفحص Virtualization: افتح Task Manager ثم Performance ثم CPU، وابحث عن Virtualization. إذا كانت Disabled، فعّلها من UEFI/BIOS حسب نوع الجهاز.

2. تثبيت WSL إذا لم يكن موجودًا

افتح PowerShell كمسؤول: من Start اكتب PowerShell ثم اضغط Right Click واختر Run as administrator. بعدها نفذ:

```
wsl --install
```

إذا طلب Windows إعادة تشغيل الجهاز، أعد التشغيل. هذا الأمر يثبت WSL وغالبًا يثبت Ubuntu تلقائيًا على الإصدارات الحديثة.

2.1 تحديث WSL وجعل WSL2 افتراضيًا

```
wsl --update  
wsl --set-default-version 2
```

2.2 التأكد من حالة WSL

```
wsl --status  
wsl --list --verbose
```

إذا ظهر Ubuntu وكانت 2 = VERSION فالإعدادات جيدة. إذا لم يظهر Ubuntu، ثبته يدويًا كما في الخطوة التالية.

2.3 تثبيت Ubuntu يدويًا عند الحاجة

```
wsl --list --online  
wsl --install -d Ubuntu  
# أو إذا كانت متاحة:  
wsl --install -d Ubuntu-24.04
```

يمكن أيضًا تثبيت Ubuntu من Microsoft Store بالبحث عن Ubuntu ثم الضغط على Install.

2.4 تشغيل Ubuntu لأول مرة وإنشاء username و password

افتح Ubuntu من Start Menu. سيطلب منك username و password. اكتب username بحروف صغيرة وبدون مسافات. كلمة السر لا تظهر أثناء الكتابة، وهذا طبيعي في Linux.

```
Enter new UNIX username: student  
New password:  
Retype new password:
```

2.5 تحويل Ubuntu إلى WSL2 إذا ظهرت Version 1

```
wsl --set-version Ubuntu 2
```

إذا كان اسم التوزيع مختلفًا، استخدم الاسم الظاهر في أمر `wsl --list --verbose`.

3. تجهيز Ubuntu بالأدوات الأساسية

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl wget nano vim iproute2 iputils-ping net-tools traceroute ca-
certificates gnupg lsb-release
```

اختبر الأدوات:

```
git --version
curl --version
ip -V
ping -V
```

4. تثبيت Docker Engine داخل Ubuntu

Docker هو المسؤول عن تشغيل Containers. وContainerlab يستخدم Docker لتشغيل النودات وربطها.

4.1 إضافة مستودع Docker الرسمي

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/
keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo ${UBUNTU_CODENAME:-
$VERSION_CODENAME}) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

4.2 تثبيت Docker packages

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

4.3 تشغيل Docker

جرب أولاً systemd:

```
sudo systemctl enable --now docker
sudo systemctl status docker
```

إذا لم يعمل systemctl داخل WSL، استخدم:

```
sudo service docker start
sudo service docker status
```

4.4 استخدام Docker بدون sudo

```
sudo usermod -aG docker $USER
newgrp docker
```

يفضل إغلاق Ubuntu وفتحه من جديد بعد إضافة المستخدم إلى مجموعة docker.

4.5 اختبار Docker

```
docker --version
docker ps
docker run hello-world
```

ظهور رسالة Hello from Docker يعني أن Docker يعمل بنجاح.

5. تثبيت Containerlab

```
bash -c "$(curl -sL https://get.containerlab.dev)"
```

اختبار Containerlab:

```
containerlab version
# أو
clab version
```

6. مفاهيم أساسية

المفهوم	المعنى
Image	قالب يحتوي نظامًا ومكتبات، مثل <code>python:3.12-alpine</code> أو <code>alpine:latest</code> .
Container	نسخة شغالة من Image ويمكن اعتبارها جهازًا وهميًا خفيفًا.
Node	أي طرف داخل الشبكة: <code>client</code> أو <code>server</code> أو <code>router</code> أو <code>switch</code> .
Server	النود التي تقدم خدمة وتستقبل اتصالات.
Client	النود التي تطلب خدمة أو ترسل بيانات للسيرفر.
Link	وصلة افتراضية بين نودين.
Topology	وصف الشبكة داخل ملف <code>YAML</code> .

7. إنشاء أول Topology بملف YAML

```
cd ~
mkdir -p network-project
```

```
cd network-project
nano lab.clab.yml
```

ضع داخل الملف:

```
name: network-project
topology:
  nodes:
    client1:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 10.0.1.11/24 dev eth1
        - ip link set eth1 up
    client2:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 10.0.2.12/24 dev eth1
        - ip link set eth1 up
    server:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 10.0.1.100/24 dev eth1
        - ip link set eth1 up
        - ip addr add 10.0.2.100/24 dev eth2
        - ip link set eth2 up
  links:
    - endpoints: ["client1:eth1", "server:eth1"]
    - endpoints: ["client2:eth1", "server:eth2"]
```

للحفظ في nano: اضغط Ctrl + O ثم Enter ثم Ctrl + X.

مهم جدًا: YAML حساس للمسافات. لا تستخدم Tab، واستخدم مسافات فقط.

8. Deploy وفحص النودات

```
sudo containerlab deploy -t lab.clab.yml
sudo containerlab inspect -t lab.clab.yml
docker ps
```

اختبر client1:

```
docker exec -it clab-network-project-client1 sh
ip addr
ping 10.0.1.100
exit
```

اختبر client2:

```
docker exec -it clab-network-project-client2 sh
ip addr
ping 10.0.2.100
exit
```

9. تجربة خدمة HTTP بسيطة على السيرفر

ادخل إلى نود السيرفر:

```
docker exec -it clab-network-project-server sh
```

داخل السيرفر:

```
apk update
apk add --no-cache python3
mkdir -p /www
echo "Hello from Containerlab server" > /www/index.html
python3 -m http.server 8080 --directory /www --bind 0.0.0.0
```

من Terminal آخر، اختبر من client1 ثم client2:

```
docker exec -it clab-network-project-client1 sh
wget -qO- http://10.0.1.100:8080
exit

docker exec -it clab-network-project-client2 sh
wget -qO- http://10.0.2.100:8080
exit
```

10. مثال عملي: client1 يرسل رسالة إلى client2 عبر server

هذا المثال يبني Message Relay Server بلغة Python. الفكرة: client2 يبقى متصلاً بالسيرفر، و client1 يرسل رسالة إلى client2 عبر السيرفر.

10.1 تجهيز مجلد المثال

```
cd ~/network-project
sudo containerlab destroy -t lab.clab.yml --cleanup
mkdir -p message-demo/server message-demo/client
cd message-demo
```

10.2 ملف server.py

```
nano server/server.py
```

```
import socket
import threading

HOST = "0.0.0.0"
PORT = 5000
```

```

clients = {}
lock = threading.Lock()

def handle_client(conn, addr):
    name = None
    try:
        first = conn.recv(1024).decode().strip()
        if not first.startswith("NAME:"):
            conn.close()
            return

        name = first.split(":", 1)[1]
        with lock:
            clients[name] = conn

        print(f"{name} connected from {addr}")

        while True:
            data = conn.recv(4096)
            if not data:
                break

            text = data.decode().strip()
            if text.startswith("T0:"):
                parts = text.split(":", 2)
                if len(parts) == 3:
                    target = parts[1]
                    message = parts[2]

                    with lock:
                        target_conn = clients.get(target)

                    if target_conn:
                        target_conn.sendall(f"FROM {name}: {message}\n".encode())
                        conn.sendall(f"DELIVERED to {target}\n".encode())
                    else:
                        conn.sendall(f"ERROR: {target} is not online\n".encode())
            finally:
                if name:
                    with lock:
                        clients.pop(name, None)
                    conn.close()

def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(10)
    print(f"Message server listening on {HOST}:{PORT}")

    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    main()

```

```
nano client/listener.py
```

```
import os
import socket
import time

NODE_NAME = os.environ.get("NODE_NAME", "client")
SERVER_IP = os.environ.get("SERVER_IP", "127.0.0.1")
SERVER_PORT = int(os.environ.get("SERVER_PORT", "5000"))

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((SERVER_IP, SERVER_PORT))
        s.sendall(f"NAME:{NODE_NAME}\n".encode())
        print(f"{NODE_NAME} connected to server {SERVER_IP}:{SERVER_PORT}")

        while True:
            data = s.recv(4096)
            if not data:
                break
            print(data.decode().strip(), flush=True)
    except Exception as e:
        print(f"Connection error: {e}")
        time.sleep(2)
```

```
nano client/send_message.py
```

```
import sys
import socket

if len(sys.argv) < 5:
    print("Usage: python send_message.py <server_ip> <sender_name> <target_name> <message>")
    sys.exit(1)

server_ip = sys.argv[1]
sender = sys.argv[2]
target = sys.argv[3]
message = " ".join(sys.argv[4:])

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((server_ip, 5000))
s.sendall(f"NAME:{sender}\n".encode())
s.sendall(f"TO:{target}:{message}\n".encode())
reply = s.recv(4096).decode().strip()
print(reply)
s.close()
```



```
nano server/Dockerfile
```

```
FROM python:3.12-alpine
WORKDIR /app
COPY server.py .
EXPOSE 5000
CMD ["python", "server.py"]
```

```
nano client/Dockerfile
```

```
FROM python:3.12-alpine
WORKDIR /app
COPY listener.py .
COPY send_message.py .
CMD ["python", "listener.py"]
```

10.6 بناء الصور

```
docker build -t msg-server:latest ./server
docker build -t msg-client:latest ./client
```

10.7 ملف lab-message.clab.yml

```
nano lab-message.clab.yml
```

```
name: msg-lab
topology:
  nodes:
    client1:
      kind: linux
      image: msg-client:latest
      env:
        NODE_NAME: client1
        SERVER_IP: 10.0.1.100
      exec:
        - ip addr add 10.0.1.11/24 dev eth1
        - ip link set eth1 up
    client2:
      kind: linux
      image: msg-client:latest
      env:
        NODE_NAME: client2
        SERVER_IP: 10.0.2.100
      exec:
        - ip addr add 10.0.2.12/24 dev eth1
        - ip link set eth1 up
```

```
server:
  kind: linux
  image: msg-server:latest
  exec:
    - ip addr add 10.0.1.100/24 dev eth1
    - ip link set eth1 up
    - ip addr add 10.0.2.100/24 dev eth2
    - ip link set eth2 up
  links:
    - endpoints: ["client1:eth1", "server:eth1"]
    - endpoints: ["client2:eth1", "server:eth2"]
```

10.8 تشغيل المثال

```
sudo containerlab deploy -t lab-message.clab.yml
docker ps
docker logs -f clab-msg-lab-client2
```

من Terminal آخر أرسل رسالة من client1 إلى client2:

```
docker exec -it clab-msg-lab-client1 python /app/send_message.py 10.0.1.100 client1 client2
"Hello client2 from client1"
```

المتوقع في client1:

```
DELIVERED to client2
```

والمتوقع في logs الخاصة بـ client2:

```
FROM client1: Hello client2 from client1
```

11. إيقاف الالابات والتنظيف

```
cd ~/network-project
sudo containerlab destroy -t lab.clab.yml --cleanup

cd ~/network-project/message-demo
sudo containerlab destroy -t lab-message.clab.yml --cleanup

docker ps
```

12. مشاكل شائعة وحلول سريعة

المشكلة	الحل
docker: permission denied	نفذ <code>sudo usermod -aG docker \$USER</code> ثم افتح Ubuntu من جديد.
Cannot connect to Docker daemon	شغل Docker باستخدام <code>sudo systemctl enable --now docker</code> أو <code>sudo service docker start</code> .

أعد تثبيت Containerlab باستخدام أمر curl الرسمي.	containerlab: command not found
تأكد أنك داخل مجلد ملف lab.clab.yml أو استخدم <code>-t</code> مع المسار الصحيح.	No topology files found
افحص <code>ip addr</code> داخل كل node وتأكد من links في YAML.	Ping لا يعمل
استخدم <code>docker ps</code> لمعرفة الاسم الكامل.	اسم الكونتينر غير معروف
افحص المسافات. YAML لا يقبل Tab ويحتاج indentation صحيح.	YAML error

13. Checklist قبل العرض

- Ubuntu يفتح بدون مشاكل.
- docker run hello-world يعمل.
- containerlab version يظهر رقم الإصدار.
- Deploy يعمل بدون أخطاء.
- docker ps يظهر server و clients.
- Ping من client1 إلى server ناجح.
- Ping من client2 إلى server ناجح.
- مثال الرسائل يعمل: client1 يرسل و client2 يستقبل عبر server.

14. مراجع رسمية مفيدة

- Microsoft WSL installation documentation: <https://learn.microsoft.com/windows/wsl/install>
- Ubuntu on WSL: <https://ubuntu.com/wsl>
- Docker Engine on Ubuntu documentation: <https://docs.docker.com/engine/install/ubuntu/>
- Containerlab installation documentation: <https://containerlab.dev/install/>
- Containerlab topology documentation: <https://containerlab.dev/manual/topo-def-file/>

Prepared by: izzaldeen mansour - انتهى الدليل